

```

718     """Conservative availability certificate over one linear-motion interval.
719
720     The moving endpoint follows the straight segment moving0->moving1. In plan view,
721     every instantaneous radio ray lies inside a narrow swept wedge around the ray to
722     the midpoint endpoint. DEM cells whose rectangles can intersect that wedge are
723     conservatively collected using a cell-center distance bound. For every collected
724     cell, a conservative ray-parameter interval gives a lower bound on line altitude.
725     This matches the nearest-cell, piecewise-constant DEM model used elsewhere.
726     """
727
728     lon_f, lat_f, alt_f = fixed
729     p_mid = tuple((a + b) / 2.0 for a, b in zip(moving0, moving1))
730     x0, y0 = _local_xy_m(moving0[0], moving0[1], lon_f, lat_f)
731     x1, y1 = _local_xy_m(moving1[0], moving1[1], lon_f, lat_f)
732     xm, ym = _local_xy_m(p_mid[0], p_mid[1], lon_f, lat_f)
733     endpoint_shift = math.hypot(x1 - x0, y1 - y0)
734     pixel_x = EARTH_R * math.cos(math.radians(lat_f)) * math.radians(dem.dx)
735     pixel_y = EARTH_R * math.radians(dem.dy)
736     half_diag = 0.5 * math.hypot(pixel_x, pixel_y)
737     sweep_radius = half_diag + endpoint_shift / 2.0 + 1e-6
738
739     # Candidate cells are selected in one vectorized pass over a conservative raster
740     # bounding box. The half-cell diagonal and endpoint motion dilate the midpoint
741     # ray into a swept wedge, so no nearest-neighbour DEM cell can be missed.
742     length_mid = math.hypot(xm, ym)
743     lon_pad = math.degrees(sweep_radius / max(EARTH_R * math.cos(math.radians(lat_f)), 1.0))
744     lat_pad = math.degrees(sweep_radius / EARTH_R)
745     lon_lo, lon_hi = min(lon_f, p_mid[0]) - lon_pad, max(lon_f, p_mid[0]) + lon_pad
746     lat_lo, lat_hi = min(lat_f, p_mid[1]) - lat_pad, max(lat_f, p_mid[1]) + lat_pad
747     ra, ca = dem.row_col(lon_lo, lat_hi)
748     rb, cb = dem.row_col(lon_hi, lat_lo)
749     row_lo, row_hi = max(0, min(ra, rb) - 1), min(dem.height - 1, max(ra, rb) + 1)
750     col_lo, col_hi = max(0, min(ca, cb) - 1), min(dem.width - 1, max(ca, cb) + 1)
751     row_grid, col_grid = np.meshgrid(
752         np.arange(row_lo, row_hi + 1), np.arange(col_lo, col_hi + 1), indexing="ij"
753     )
754     cell_lons = dem.lon0 + col_grid * dem.dx
755     cell_lats = dem.lat0 - row_grid * dem.dy
756     cell_x = EARTH_R * math.cos(math.radians(lat_f)) * np.radians(cell_lons - lon_f)
757     cell_y = EARTH_R * np.radians(cell_lats - lat_f)
758     denom = max(length_mid * length_mid, 1e-18)
759     r_center_all = np.clip((cell_x * xm + cell_y * ym) / denom, 0.0, 1.0)
760     qx, qy = r_center_all * xm, r_center_all * ym
761     dist_all = np.hypot(cell_x - qx, cell_y - qy)
762     # 0.5 m absorbs the local equirectangular vectorisation approximation.
763     mask = dist_all <= sweep_radius + 0.5
764     candidate_rows = row_grid[mask]
765     candidate_cols = col_grid[mask]
766     candidate_r = r_center_all[mask]
767
768     los_certified = True
769     min_clearance_lb = float("inf")
770     plan_lengths = [math.hypot(x0, y0), math.hypot(x1, y1)]
771     length_floor = max(1e-9, min(plan_lengths))
772     r_expand = min(1.0, sweep_radius / length_floor)
773     triangle_det = x0 * y1 - x1 * y0
774     static_plan = endpoint_shift <= 1e-6
775     if candidate_rows.size and abs(triangle_det) > 1e-3:
776         # The union of all instantaneous rays is exactly triangle F-M0-M1.
777         # Since both plan position and altitude are affine in time, radio-ray
778         # altitude is an affine plane over this triangle. Clip every DEM cell
779         # rectangle against the triangle and test the plane only on the clipped
780         # polygon vertices, where its minimum must occur.
781         plane = np.linalg.solve(
782             np.asarray([[0.0, 0.0, 1.0], [x0, y0, 1.0], [x1, y1, 1.0]]),
783             np.asarray([alt_f, moving0[2], moving1[2]], dtype=float),
784         )
785         triangle = [(lon_f, lat_f), (moving0[0], moving0[1]), (moving1[0], moving1[1])]
786         clearances: list[float] = []
787         for row, col in zip(candidate_rows.tolist(), candidate_cols.tolist()):
788             center_lon = dem.lon0 + col * dem.dx
789             center_lat = dem.lat0 - row * dem.dy
790             clipped = _clip_polygon_rect(
791                 triangle,
792                 center_lon - dem.dx / 2.0,
793                 center_lon + dem.dx / 2.0,
794                 center_lat - dem.dy / 2.0,

```